山东大学 计算机科学与技术 学院

机器学习 课程实验报告

学号：202200130077	姓名： 薛润泽	邮箱：
实验题目：Experiment 10: Random Forest		

实验过程：

（记录实验过程、遇到的问题和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

1、随机森林算法：随机森林算法（Random Forest）是集成学习（Ensembel Learning）算法中的一种。它主要思路是通过构建多个决策树，并通过这多个决策树进行样本点预测，并利用类似于“专家投票”的思想，将多个决策树的结果结合起来，从而减少过拟合的可能性，提高预测的准确性。

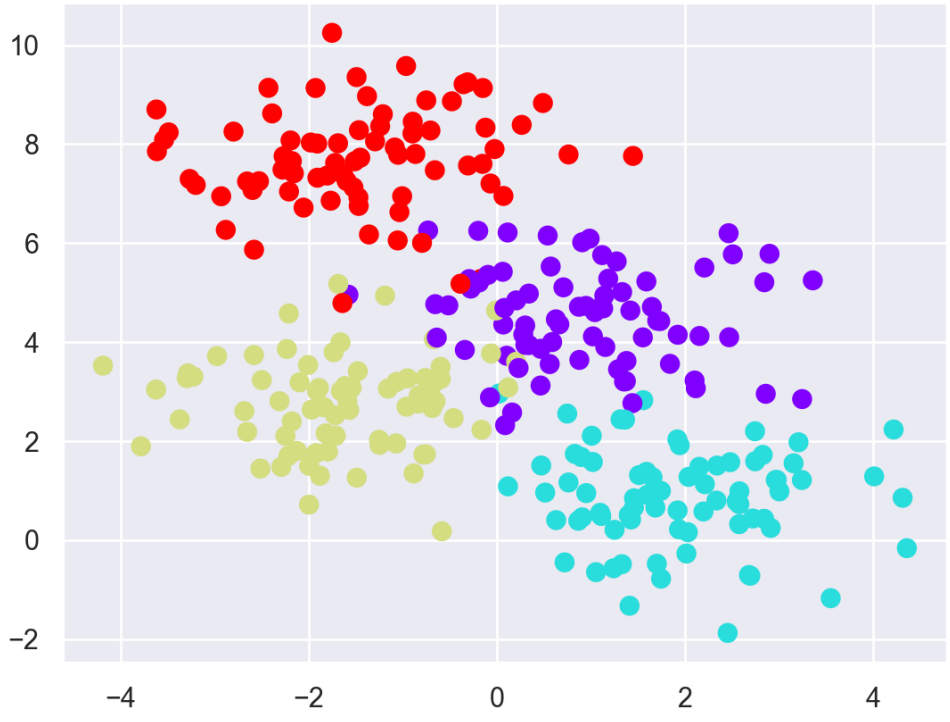
2、数据生成

本次实验首先讨论了决策树的构建，因为二进制拆分会使得决策树分类高效（期望上一次减少一半选项），这样的二进制拆分如果体现在图形上可以是轴对齐拆分的形式（在整体数据图中画一条直线，将不同类分开）

通过下面代码进行数据生成：

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples = 300, centers = 4, random_state = 0,
cluster_std = 1.0)
plt.scatter(X[:, 0], X[:, 1], c = y, s = 50, cmap ='rainbow')
```

获得数据如下图：



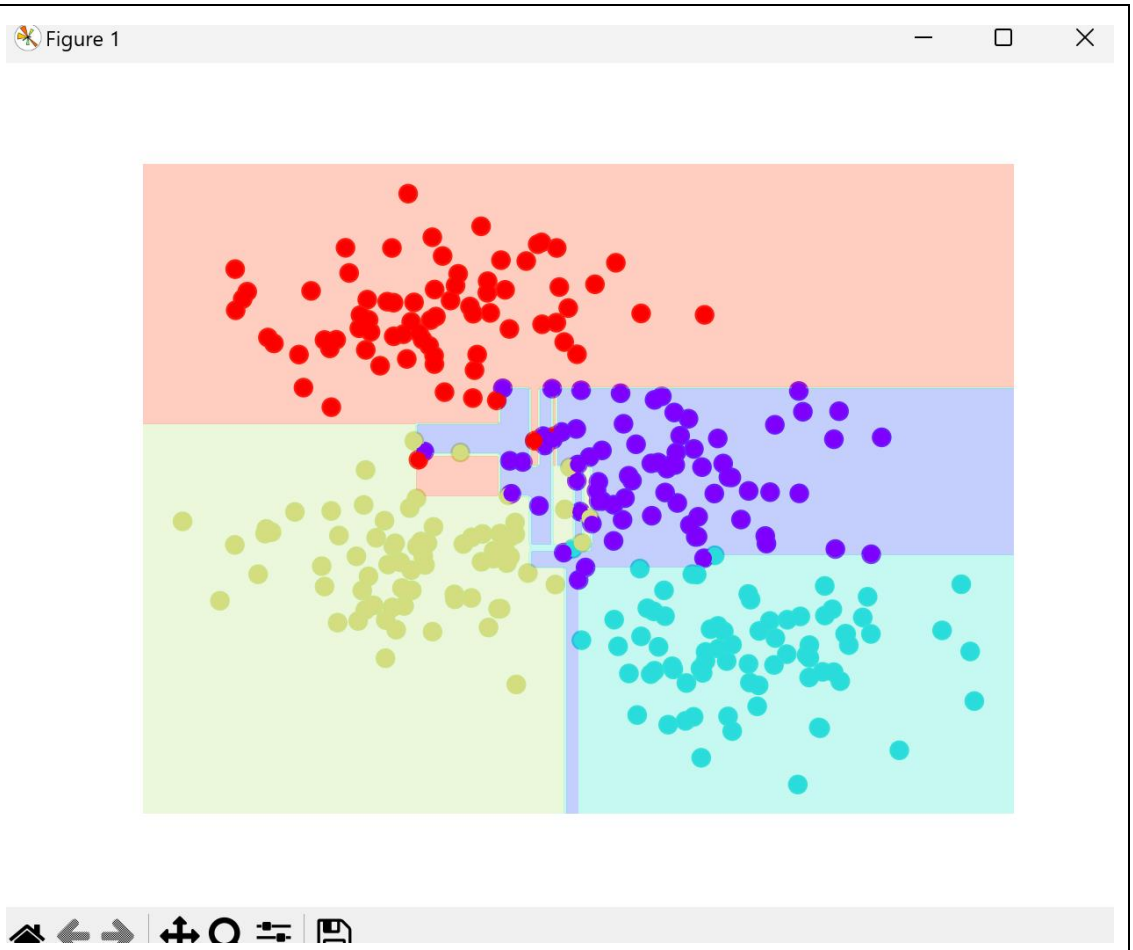
3、决策树建立

使用下面的代码建立决策树，并定义函数完成分类器的可视化：

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier().fit(X, y)
```

```
def visualize_classifier(model ,X, y, ax = None, cmap = 'rainbow'):
    ax = ax or plt.gca()
    #Plot training points
    ax.scatter(X[:, 0], X[:, 1], c = y, s = 30, cmap = cmap, clim =
(y.min(), y.max()), zorder = 3)
    # ax.scatter(X[:, 0], X[:, 1], c = y, s = 30, cmap = cmap, zorder = 3)
    ax.axis('tight')
    ax.axis('off')
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
    #fit the estimator
    model.fit(X, y)
    xx, yy = np.meshgrid(np.linspace(*xlim, num = 200), np.linspace(*ylim,
num = 200))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape((xx.shape))
    # Create a color plot with the results
    n_classes = len(np.unique(y))
    contours = ax.contourf(xx, yy, Z, alpha = 0.3, levels =
np.arange(n_classes + 1) - 0.5, cmap = cmap, clim = (y.min(), y.max()),
zorder = 1)
    # contours = ax.contourf(xx, yy, Z, alpha = 0.3, levels =
np.arange(n_classes + 1) - 0.5, cmap = cmap, zorder = 1)
    ax.set(xlim = xlim, ylim = ylim)
```

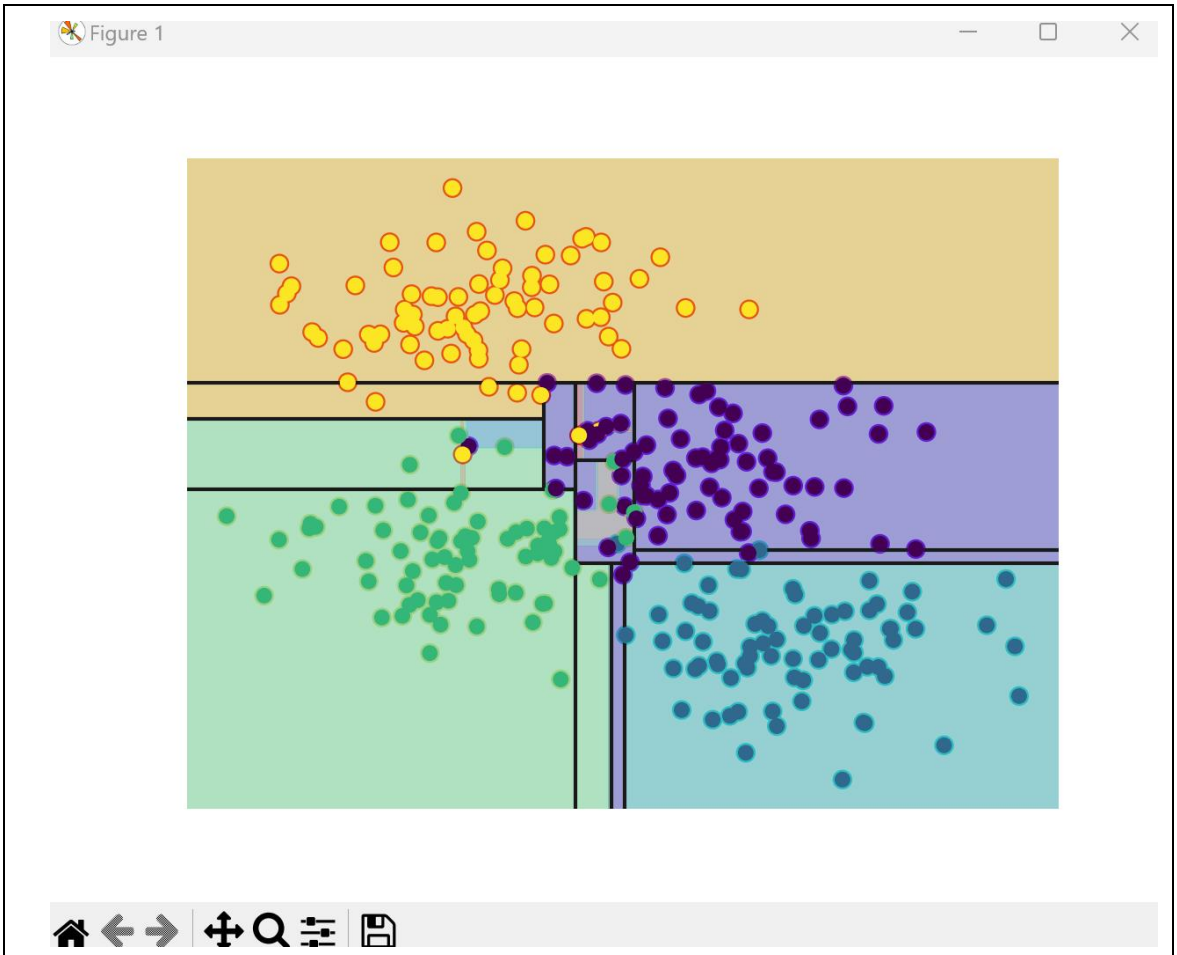
完成决策树构建后，调用可视化函数并画图，得到结果如下图：



使用下面的代码进行交互式可视化：

```
import helpers_05_08
helpers_05_08.plot_tree_interactive(X, y)
```

画出图像如下：



通过观察上述图像可以发现，中间多类别交界处出现了不符合预期的状态。以绿色点区域为例子，因为在绿色区域中存在红色点和深蓝色点，所以区域划分时出现了绿色区域中包围蓝色和红色区域的状况。我们预期的实际颜色区域应该如交互式图像中显示的黑色线条分割区域，在绿色区域中的红色点和深蓝色点可以看做是噪声点。所以我们构建的上述决策树发生了过拟合现象。

一颗决策树很容易过拟合，构建决策树时容易拟合特定数据而不是对整体数据进行提取。所以可以通过不同数据集训练多颗不同的树，使用多颗树的信息来更清晰的确定分类不明确地方。

使用下述代码构建随机决策树

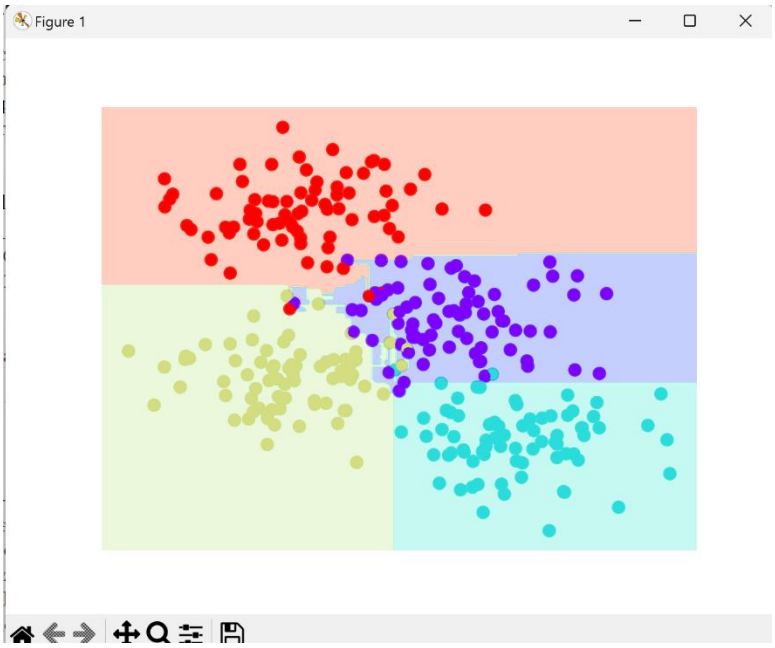
```
import helpers_05_08
helpers_05_08.randomized_tree_interactive(X, y)
```

4、使用 Bagging 的方法来训练集成模型，80%的训练点的随机子集来随机化数据。代码如下：

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

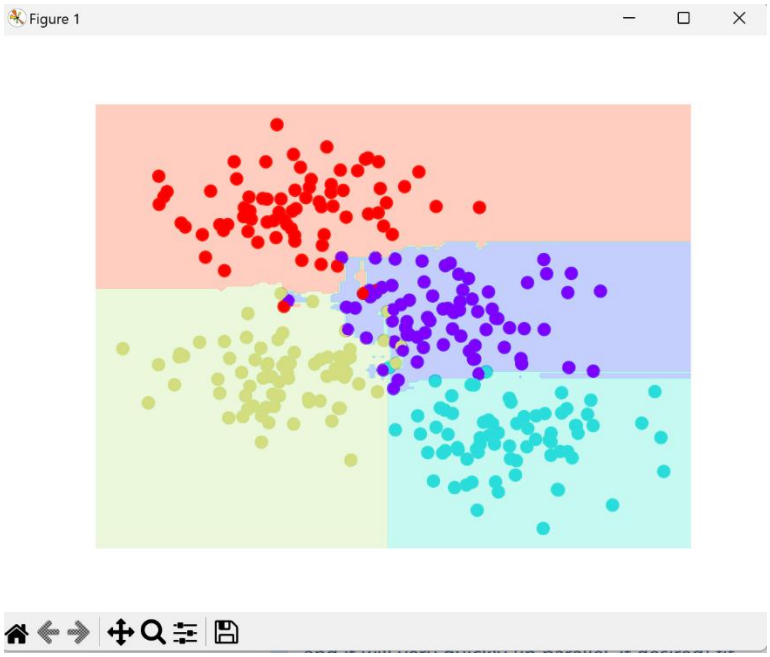
tree = DecisionTreeClassifier()
bag = BaggingClassifier(tree, n_estimators = 100, max_samples = 0.8,
random_state = 1)
bag.fit(X, y)
visualize_classifier(bag, X, y)
```

画图如下图：



5、使用下面的代码创建随机决策树森林，并画图

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators = 100, random_state = 0)
visualize_classifier(model, X, y)
```



结果分析与体会：

1、决策树容易造成过拟合的现象，为了解决这个现象我们可以使用集成学习的方法构建多棵决策树来进行分类。

2、随机的方法来选取数据可以更好的帮助构建决策树